

Design and Implementation of Configuration Memory SEU-Tolerant Viterbi Decoders in SRAM-Based FPGAs

Zhen Gao¹, Jinhua Zhu, Ruishi Han, Zhan Xu, Anees Ullah², and Pedro Reviriego³

Abstract—A Viterbi decoder is used in many communication receivers to efficiently decode the received signal that has been convolutional encoded in the transmitter. This decoding corrects errors that occur due to noise and other imperfections in the channel and is key to achieve a low bit error rate. If the decoder is implemented on a SRAM-based field-programmable gate array (SRAM-FPGA), the radiation-induced soft errors can affect the operation of the Viterbi decoder by corrupting the configuration memory, which can change the circuit functionality and will not be corrected unless the FPGA is reconfigured. This makes the protection of Viterbi decoders implemented on SRAM-based FPGAs an important issue. In this paper, first, fault injection experiments are conducted to study the effects of soft errors on the configuration memory of an SRAM-FPGA implemented Viterbi decoder. Then, an efficient protection technique is proposed and evaluated showing that it can provide an efficient protection with a lower resource usage than the traditional triple modular redundancy approach.

Index Terms—Viterbi decoding, FPGA, fault tolerance, single event upset, duplication with comparison.

I. INTRODUCTION

MANY modern communication systems, rely on convolution coding combined with Viterbi decoding to achieve the desired performance at low signal to noise ratios [1]–[4]. Depending on the application, the decoding algorithm can be implemented in software running on a Digital Signal Processor (DSP), on an Application Specific Integrated Circuit (ASIC) or on a Field Programmable Gate Array (FPGA) [5]. In all cases, the implementation can be affected by radiation induced soft errors that can alter the correct operation of the decoder [6]. One of such example is space communication with on-board

digital signal processing (OBP). With the increasing demand of modern mobile satellite communication systems for high communication quality and user capacity, the conventional transparent transponder becomes insufficient, and the OBP gets more attention due to its flexibility and good performance [7], [8]. For example, Iridium and Thuraya mobile satellite communication systems apply Viterbi codec in their OBPs [9], [10]. In addition to strong processing capability and large capacity, static random-access memory based FPGAs (SRAM-FPGAs) are also very flexible for reconfiguration, which makes the online upgrade of the OBP function possible, so SRAM-FPGAs are considered as good candidates for OBP platforms [11]–[14]. However, SRAM-FPGAs are very sensitive to soft errors caused by cosmic radiation, and the single event upset (SEU) effect is typically the most frequent one [6], [11]. SEUs can corrupt both the configuration memory and the user memories (including flip-flops and Block RAM) of SRAM-FPGAs [15], [16]. While errors on user memory only corrupt the data stored, errors on the configuration memory can change the circuit implemented by the FPGA and can only be corrected by reconfiguration. This makes the protection of SRAM-FPGA based Viterbi decoder implementations against errors on the configuration memory an important issue in harsh environment like OBP based space communication.

The most popular SEU protection method is Triple Modular Redundancy (TMR) [17]–[19]. One of its advantages is that the application of TMR is independent of the logic under protection, so it can be used to protect almost any module. But the problem of TMR is that it more than triples the logic and power overhead, which may not be affordable for an OBP platform that is limited in terms of volume and power [20]. Another approach is to try to use the algorithmic properties of the circuit to detect/correct errors, which is referred to as algorithm-based fault tolerance (ABFT) [21]. This strategy can reduce the overhead required to protect a circuit. Protection for different decoders has been studied in several works using the ABFT approach. Most of these studies focus on errors on the memories used in Turbo and Low Density Parity Check (LDPC) decoders [22], [23]. For example in [22], the algorithm for decoding of Turbo codes is modified so that it is more resilient to errors on the stored input signal, while in [23], only the most significant bits are protected and more iterations are used to repair errors. For the Viterbi decoder, soft errors on user memory have been discussed in some papers, but errors on the configuration memory have not been carefully studied. In [24], the errors caused by reducing the voltage supply

Manuscript received December 10, 2018; revised April 10, 2019; accepted June 25, 2019. Date of publication July 9, 2019; date of current version July 22, 2019. This work was supported in part by the National Natural Science Foundation of China under Grant 61501321 and Grant 61620106001, in part by the Tianjin Science and Technology Program under Grant 17ZXRGX00160, and in part by the TEXEO Project (TEC2016-80339-R) funded by the Spanish Ministry of Economy and Competitiveness. The review of this paper was arranged by associate editors M. Ottavi, V. Sridharan, and S. Tragoudas. (*Corresponding author: Zhen Gao.*)

Z. Gao, J. Zhu, and R. Han are with the Tianjin University, Tianjin 300072, China (e-mail: zgao@tju.edu.cn; 1647478422@qq.com; hrs@tju.edu.cn).

Z. Xu is with the Beijing Information Science and Technology University, Beijing 100085, China (e-mail: xuzhan@bistu.edu.cn).

A. Ullah is with the Sir Syed CASE Institute of Technology, Islamabad 44000, Pakistan (e-mail: anees.ullah@case.edu.pk).

P. Reviriego is with the Universidad Carlos III de Madrid 28911 Leganés, Spain (e-mail: revirieg@it.uc3m.es).

Digital Object Identifier 10.1109/TNANO.2019.2925872

to the decoder circuitry to reduce power consumption have been considered, and the decoder was optimized by considering the combined effects of channel noise and memory errors. In [25], Error Detection and Correction (EDAC) combined with partial TMR was proposed to protect the both configuration memory and user memory. Our recent work [26] considered errors on the user memory of an FPGA based Viterbi decoder, and found that most of the SEUs can be tolerated by the decoder. This paper extends that work by considering SEUs on the configuration memory.

In this paper, we focus on the study of SEUs on the configuration memory of the Viterbi decoder implemented on SRAM-FPGA, including the cases of hard input and soft input decoding. First, we show by fault injection experiments that SEUs on configuration memory have a large impact on the reliability of the decoder. Then, an ABFT protection technique based on an enhanced Duplication with Comparison (DWC) structure is proposed and evaluated. The results show that it significantly reduces the resource usage compared to TMR while achieving a good error protection. It should be noted that the fault tolerant design can only prevent errors from propagating to the outputs when SEUs occur. Reconfiguration is still required to repair the fault in the SRAM-FPGA. This last part is not discussed in this paper. In addition, it is worth mentioning that the proposed scheme may also be effective to protect against SEUs on the user memory in a ASIC implementation.

The rest of the paper is structured as follows. Section II presents the principle of convolutional codes and Viterbi decoding. A practical Viterbi decoder is presented in section III that also evaluates the effect of SEUs on the unprotected decoder. The proposed protection technique is presented in Section IV and evaluated in Section V. The paper ends with the conclusions and some ideas for future work in section VI.

II. OVERVIEW OF THE VITERBI DECODER

In this section, the principle and basic implementation structure of a Viterbi decoder are introduced with a simple example. The decoder implemented for evaluation will be introduced in the next section.

A. Convolutional Codes

A convolutional code is usually expressed as (n, k, m) , where m denotes the number of shift registers in the encoder, and k and n denote the number of input bits and output bits for the encoder in each step, respectively. An input sequence with N bits should be segmented into L groups with k bits in each group. For example, for a $(2, 1, 2)$ convolutional code encoder, 1 bit is input ($k = 1$) to generate 2 bits ($n = 2$) for each step, and there are 2 shift registers in the encoder ($m = 2$). The specific processing of an encoder is determined by the generation matrix. For a convolutional code $(2, 1, 2)$, the generation matrix is $G = [1 \ 1 \ 1; 1 \ 0 \ 1]$ or expressed as octal number pair of $(7,5)_8$, whose structure is as shown in Fig. 1(a). The behavior of the encoder can be expressed by the state transition diagram in Fig. 1(b), in which solid lines and dashed lines stand for an input bit of 0 or 1, respectively, and the bit pair on the line are the corresponding encoded bits. Usually,

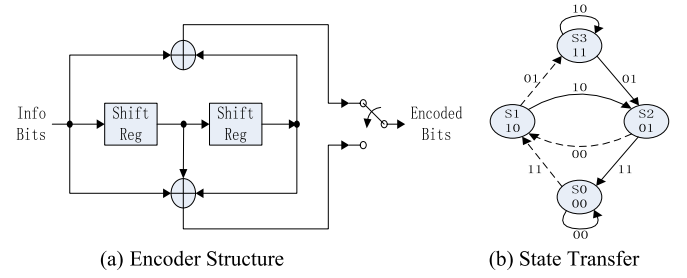


Fig. 1. Encoder of the $(2, 1, 2)$ convolutional code.

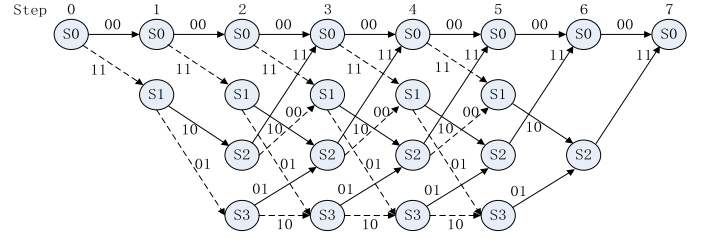


Fig. 2. Trellis diagram of the $(2, 1, 2)$ convolutional code.

an encoder starts with state S_0 , and $m \cdot k$ zeros are added after the original raw bits so that the encoder will return to state S_0 . So the length of the input bit sequence is $(L + m) \cdot k$, and the encoded sequence length is $(L + m) \cdot n$.

By expanding the state transition diagram over steps, the whole procedure of the encoder can be expressed by the Trellis diagram ($L = 5$) as shown in Fig. 2. For a convolution code of (n, k, m) , the number of states is 2^{km} , and there are 2^k branches entering and leaving each state. The number of possible paths for an input sequence with $k \cdot L$ bits in the Trellis diagram is 2^{kL} .

B. Viterbi Decoding

For a received signal $R = [r_1, r_2, \dots, r_{nL}]$, a maximum likelihood decoder (MLD) tries to find a path (or a codeword C) in the Trellis diagram among the 2^{kL} possible paths so that the likelihood $P(R|C)$ is maximized, which can be expressed as:

$$\max_j \log P(R|C_j), j = 1, 2, \dots, 2^{kL} \quad (1)$$

For a binary symmetric channel (BSC) (R is changed to a bit sequence by hard decision as R_H), equation (1) is equivalent to minimizing the Hamming distance between R_H and C , which can be expressed as

$$\max_j d(R_H, C_j), j = 1, 2, \dots, 2^{kL} \quad (2)$$

In this case, C is also a bit sequence. To improve the decoding, soft information is usually taken as the input instead of a bit sequence. Soft information of a received symbol r is usually measured by the log-likelihood ratio (LLR) of binary symbol $c_i = +1$ over $c_i = -1$, which can be expressed as

$$L(r_i) = \ln \left[\frac{p(r_i|c_i = +1)}{p(r_i|c_i = -1)} \right], \quad (3)$$

which can be further simplified under an Additive White Gaussian Noise (AWGN) channel as $L(r_i) = L_c r_i$, where

$L_c = 4E_s = N_0$ is defined as the channel reliability factor. In this case, the target for equation (3) can be reformulated to choose a code to maximize the correlation with the received signal, which can be expressed as

$$M(R|C_j) = \sum_{i=1}^{nL} L_c c_i^j r_i, \quad (4)$$

where $C_j = [c_1^j, c_2^j, \dots, c_{nL}^j]$ is a sequence with $c_i^j = 1$.

The Viterbi decoder is designed to simplify the path search process [4]. Based on the Trellis diagram in Fig. 2, we see that there are 2 branches merging at each node after 3 transitions with path metric (PM) of PM_0 and PM_1 . Since any particular path through the trellis that stems from a specific node (e.g., S_1) will add identical terms to the path metrics, if $PM_0 > PM_1$ at the merged node S_1 , PM_0 will continue to be larger than PM_1 for any path that stems from node S_1 . This means that the path corresponding to PM_1 can be discarded from further consideration, and that the path corresponding to PM_0 is the survivor. Similarly, one of the two paths that merge at state node S_0 , S_2 or S_3 can be eliminated on the basis of the two corresponding path metrics. As a result, after the first 3 transitions, there are 4 surviving paths, one terminating at each state, and a corresponding metric (PM) for each surviving path. This procedure is repeated at each stage of the trellis as new signals are received in the subsequent time intervals. In general, for a convolutional code of (n, k, m) , there are 2^{km} states. Hence, there are 2^{km} surviving paths at each stage and 2^{km} metrics, one for each surviving path. Consequently, the decoding of such a code by means of the Viterbi algorithm requires keeping track of 2^{km} paths and 2^{km} metrics. At each stage of the trellis, there are 2^k paths that merge at each node, and only the most likely path survives. During the decoding process, the metric for each transition is defined as the branch metric (BM), and all the BM s for the transitions along a path are accumulated to produce the path metric (PM).

C. Implementation of the Viterbi Decoder

Based on the above introduction, a commonly applied implementation of a parallel Viterbi decoder for a bit input is shown in Fig. 3 (for the convolutional code (2,1,2) in Fig. 1), in which $\oplus$ means Hamming distance calculation. The input sequence is a bit stream obtained by hard decision of the received signal. The branch metric is first calculated for each state, which is accumulated later to the path metrics for the two merged paths. After comparison, the merged path with the larger metric (or smaller Hamming distance) is selected as the surviving path. These operations for each state branch can be summarized as ACS, where A means accumulation of the branch metric, C means comparison of the path metrics of the merged paths, and S means selection of the surviving path. For each state, the survivor path information is stored. Finally, a trace back procedure outputs the decoded bits based on the stored survivor path information.

For a soft input Viterbi decoder, the implementation structure is similar except that the BM is calculated by correlation and the path with the larger PM will survive.

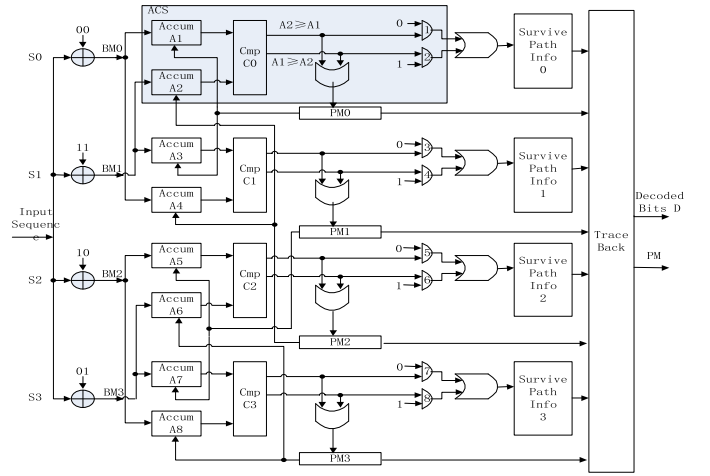


Fig. 3. Implementation structure of parallel Viterbi decoder.

TABLE I
RESOURCE USAGE OF THE UNPROTECTED VITERBI DECODER WITH HARD INPUT

	C&S	BM/PM	Traceback	BlockRam	Total
LUTs	246	1762	45	18	2071
Registers	518	293	75	0	886
BRAM	0	0	0	1	1

TABLE II
RESOURCE USAGE OF THE UNPROTECTED VITERBI DECODER WITH SOFT INPUT

	C&S	BM/PM	Traceback	BlockRam	Total
LUTs	413	3615	45	18	4091
Registers	838	1701	75	0	2614
BRAM	0	0	0	1	1

III. FAULT TOLERANCE EVALUATION OF THE UNPROTECTED VITERBI DECODER

A. FPGA Implemented Viterbi Decoder for Evaluation

A convolutional code (3,1,6) with generation matrix $G = (133, 173, 165)_8$ is implemented for evaluation in this paper. This code is used in the Long Term Evolution (LTE) standard for the Physical Downlink Control Channel (PDCCH) [27]. The Viterbi decoder is implemented in HDL and mapped to a Xilinx series 7 FPGA hosted in a Zedboard for hard and soft inputs. The basic structure is same as that introduced in Section II-C, and the main implementation differences include: 1) the number of parallel ACS modules increases from 4 to 64, each of which corresponds to a state among 000000 to 111111; 2) the table length for LUTs based BM calculation changes from 4 to 8, corresponding the distance from 0 (000) to 7 (111); 3) the number of bits for surviving path information is enlarged 16 times (64 states over 4 states); 4) the width of shift registers in the traceback module increases from 2-bits to 6-bits. The decoder is composed of four parts: BM and PM calculation, Comparison and Selection (C&S), Traceback and BlockRam. The resource usage of the Viterbi decoder with hard and soft inputs are listed in Table I and II, respectively. In the experiments, the length of

TABLE III
FAILURE RATE FOR THE UNPROTECTED VITERBI DECODER WITH HARD INPUT

BER	0	6.67%	10%	20%
Failure rate	4.7%	12.6%	13.9%	32%

TABLE IV
FAILURE RATE FOR THE UNPROTECTED VITERBI DECODER WITH SOFT INPUT

SNR	20 dB	10 dB	0 dB
Failure rate	7.4%	7.6%	11.6%

the information bits is fixed to be 60, and the total delay for the decoding is 121 cycles, among which 60 cycles are used for forward processing, 60 cycles are used for traceback and one more cycle for the output.

B. Evaluation of the Fault Tolerance of the Decoder

SEUs have been injected on the configuration memory using an adapted version of the fault injection tool in [28]. In this section we only present the fault injection results to show the SEUs tolerance capability of the decoder, and leave the introduction of the platform and procedure for fault injection experiments to Section V-B. Based on our implementations, the numbers of essential bits [29] in the configuration memory for hard and soft decoding are 380251 and 753918, respectively. One hundred different inputs are tested for the upset of each bit, which is then labeled as a failure bit (or critical bit defined in [29]) if at least one of the 100 outputs is different from that of the golden decoder. For hard decoding, the inputs are generated randomly under different bit error rates (BER = 0, 6.67%, 10% or 20%). For soft decoding, the inputs are generated under different input signal to noise ratios (SNR = 0 dB, 10 dB or 20 dB). A BER of 20% and a SNR of 0 dB are around the decoding thresholds for hard and soft decoding, respectively, obtained by simulation tests.

The failure rates (fraction of bits causing failures over the total essential bits) for hard decoding and soft decoding are summarized in Tables III and IV, respectively. It is interesting to observe that the failure rate is related to the signal quality. For hard decoding, the failure rate is only 4.7% for a BER of 0 and increases to 32% for a BER of 20%. For soft decoding, the failure rate is around 7.5% for high SNR, and increases to 11.6% for a SNR of 0 dB. Considering that the critical bits typically account for about 1/3 of the essential bits as reported in [30], we can conclude that: 1) for an input with low BER or high SNR, most of the SEUs on configuration memory could be tolerated by the decoder itself; 2) for an input with high BER or low SNR, SEUs on large portion of the critical bits would cause decoding failure. This means that since the signal quality is determined by the channel and cannot be guaranteed in a real application, there is a strong need for protection against SEUs on configuration memory in practical implementations.

IV. PROPOSED FAULT TOLERANT VITERBI DECODER

In this section, first, the failures caused by SEUs on configuration memory are analyzed and then used to explain the fault

injection results. Then, an efficient protection technique is proposed based on the analysis.

A. Fault Analysis

Based on the previous description of the decoder, the logic function of Viterbi decoder can be generally divided into four parts: C&S, BM and PM calculation, trace back and Block RAM. A fault on each part may cause wrong decoding results as follows:

- 1) The C&S logic is corrupted so that a wrong path will be selected for the corresponding state. Such failure is not related to the signal quality, but may not cause wrong outputs if the state is not on the expected path. In this case, the PM corresponding to the wrong path must be smaller than that of the expected path.
- 2) The calculation of PM/BM is wrong. The wrong value of PM or BM may not result in a wrong decoding output, and the effect is related to the signal quality and the fault situation, so the analysis for this part is divided in four cases:
 - a) The PM for the expected path increases;
 - b) The PM for an unexpected path decreases;
 - c) The PM for the expected path decreases;
 - d) The PM for an unexpected path increases;

For cases a) and b), the expected path will survive, so the fault will not change the decoding result. For case c), if the PM for the expected path is corrupted to be smaller than that of another path, a wrong path will be selected, which will cause a wrong decoding result. In this case, the final PM should be smaller than the expected value. For case d), the PM for an unexpected path may be increased to be larger than that of the expected path, so that a wrong path is selected with a larger PM than the expected value. For the latter two cases, whether the fault would cause an error outputs depends on the difference between the PM for the wrong path and that for the expected path before and after the corruption. For a high signal quality, the PM for the expected path would be much larger than that for the other paths, so the possibility for wrong path selection is low. On the other hand, the gap between the PM for the expected path and that for other paths is small for low quality signal, so it is much easier to change the path selection.

- 3) A fault occurs during the trace back process, which may cause a wrong decoding result. This failure is not related to the signal quality. In this case, the final PM is the expected value, but different from that corresponding to the output sequence.
- 4) The Block RAM is used to store the surviving path information that would be used during the trace back process. A SEU on this part will cause wrong outputs of trace back. Similar to case (3), the final PM is correct but different from that corresponding to the decoding results.

In summary, SEUs on part (1), (3) and (4) may cause wrong decoding results, and the effect of those on part (2) depends on different situations and is related to the signal quality. Based on this analysis, the evaluation results on unprotected Viterbi decoder can be explained as part of the errors depend on the signal quality.

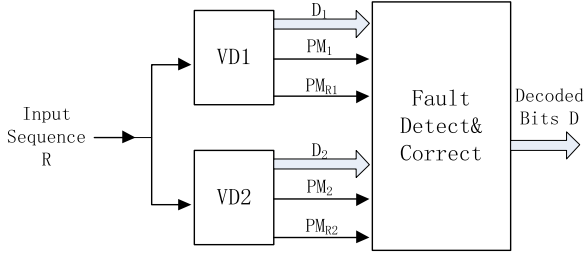


Fig. 4. Structure of the proposed fault-tolerant Viterbi decoder.

B. DWC Based Fault Tolerant Viterbi Decoder

An enhanced DWC scheme is proposed for the protection of the Viterbi decoder against SEUs, whose structure is shown in Fig. 4. Each Viterbi decoder provides three outputs: the decoded bit stream (D), the PM value and the recalculated PM (PM_R) during the trace back process. For a hard input Viterbi decoder (HIVD), PM_R is the Hamming distance between the input bit sequence and the selected code (0/1 sequence). For a soft input Viterbi decoder (SIVD), PM_R is the correlation between the input signal and the selected code (+1/-1 sequence). The fault detection and correction module will first compare the two decoded bit streams, if they are same, one of them will be output directly. If the two outputs are different, PM and PM_R will be used to find the faulty decoder and output the decoding result of the other decoder. Therefore, error correction is implemented using only two copies of the decoder instead of the three needed in TMR. This is similar to other improved DWC schemes that have been presented for other signal processing circuits that exploit the features of the circuits to identify the erroneous copy [31].

Based on the analysis in Section IV-A, if a fault of case (1) or subtype c) in case (2) happens and causes wrong decoding results, the decoder providing the smaller PM value will be identified as the faulty decoder. But for subtype d) in case (2) and case (3) or (4), the value of the PM of the faulty decoder will be larger than and same to that of the correct one, respectively. In these two cases, we need to compare the PM and PM_R from each decoder. For the correct decoder, these two values should be the same. But for the decoder with a fault of subtype d) in case (2) and case (3) or (4), these two values will be different. This is because the recalculated PM is the measurement of the similarity between the selected code corresponding to the decoded bits and the input signal. If an unexpected code is selected, the recalculated PM must be different from that of the expected code. Finally, the fault detection and correction module will work according to the procedure shown in Fig. 5. This figure is for the case of a soft input, so the decoder with larger PM will be the correct one for the last decision. For the Viterbi decoder with hard input, the first three decisions are the same as that for SIVD, but the last decision changes to the opposite, which means that the decoder providing smaller PM will be the correct one.

The recalculation of the PM is embedded into the trace back process. In the modified implementation of trace back, for each step during the trace back, the ‘re-encoding’ related to current states is performed simultaneously, which is used to re-calculate

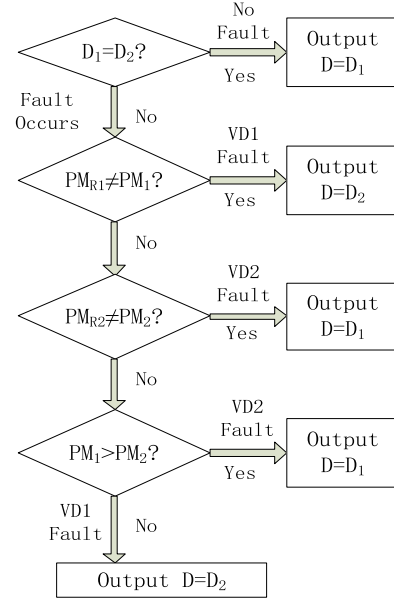


Fig. 5. Procedure of fault detection and correction (for the case of soft input).

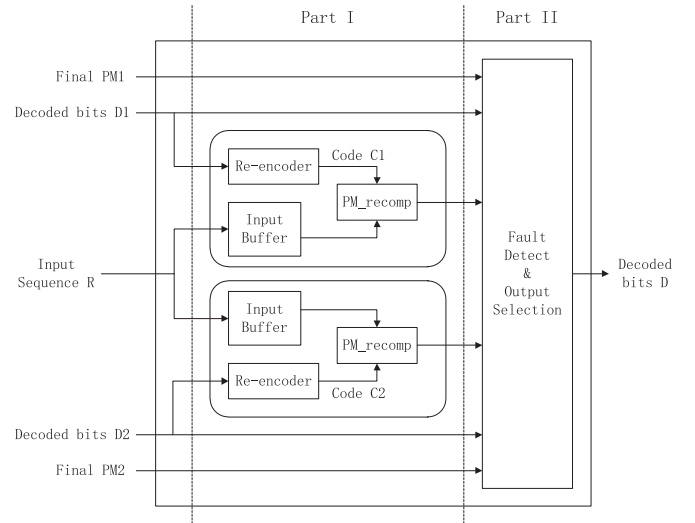


Fig. 6. Redundant parts of the proposed DWC-based Viterbi decoder.

the BM value and update the value of PM_R . The final PM_R is generated when the trace back process finishes.

It should be noted that, apart from the two normal Viterbi decoders, the proposed fault tolerant system includes two redundant elements (as shown in Fig. 6): the recalculation of the PM (Part I) and the final fault detection and output selection module (Part II). Part I includes a buffer for the input signal, a convolution encoder and a correlator, and all these three parts are embedded into the trace back procedure of each decoder in our implementation, so a fault on each part can be detected by the final fault detection procedure. Therefore, no additional protection is needed for this part. Part II needs separate protection, but it just includes three comparators according to Fig. 6, so even TMR protection would introduce a low overhead.

TABLE V
OVERHEAD EVALUATION OF THE UNPROTECTED AND PROTECTED VITERBI
DECODER WITH HARD INPUT (RATIO OVER THE UNPROTECTED CASE
IN PERCENTAGE)

	Slice LUTs	Slice Registers	Block RAMs	Extra Delay
Unprotected	2071	886	1	0
DWC Protected	4526 (218%)	1960 (221%)	2 (200%)	1
TMR Protected	6311 (305%)	2658 (300%)	3 (300%)	0

TABLE VI
OVERHEAD EVALUATION OF THE UNPROTECTED AND PROTECTED VITERBI
DECODER WITH SOFT INPUT (RATIO OVER THE UNPROTECTED CASE
IN PERCENTAGE)

	Slice LUTs	Slice Registers	Block RAMs	Extra Delay
Unprotected	4091	2614	1	0
DWC Protected	8672 (212%)	5460 (209%)	2 (200%)	1
TMR Protected	12413 (303%)	7842 (300%)	3 (300%)	0

TABLE VII
LUTs USAGE OF FAULT DETECTION AND CORRECTION MODULE

	HIVD	SIVD
DWC	143	151
TMR	122	122

V. EVALUATION OF THE PROPOSED FAULT TOLERANT VITERBI DECODER

In this section, the proposed fault tolerant DWC Viterbi decoder is evaluated and compared in terms of FPGA resource usage and fault tolerance with the TMR protected decoder.

A. Resource Overhead

The proposed protection technique has been used to protect the Viterbi decoder used as case study for both HIVD and SIVD implementations. Both designs have been implemented in HDL and mapped to a Xilinx series 7 FPGA hosted in a Zedboard. As a reference, TMR protected HIVD and SIVD are also implemented on the same board. The synthesis option of ‘resource_sharing’ is disabled to ensure that modules are independent and isolated in all the implementations.

Tables V and VI summarize the usage of LUTs, registers and BRAMs for the unprotected, DWC protected and TMR protected Viterbi decoders for hard and soft inputs, respectively. For the protected decoders, the relative cost versus the unprotected decoder is also shown in parenthesis as a percentage. From the results it can be observed that the overheads of DWC protected HIVD and SIVD are around 2.2 times and 2.1 times of that of the unprotected HIVD and SIVD, respectively. This compares favorably with TMR that has a value slightly larger than 3. Therefore, we can conclude that the proposed protection scheme is able to reduce the FPGA resource usage significantly compared to TMR. Table VII compares the LUTs as logic in the fault detection and correction module in the proposed DWC and TMR schemes, from which we can see that the fault detection and correction module in DWC is more complex than that in the TMR scheme as the operations in Fig. 5 require more

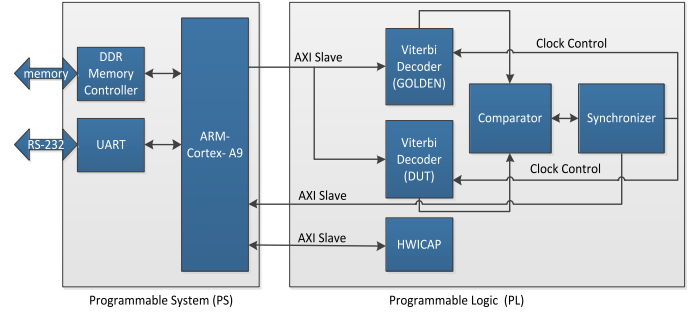


Fig. 7. Zynq-SoC based fault injection and evaluation platform.

comparisons. In terms of delay, the DWC protected Viterbi decoder just introduces an additional delay of one cycle compared with the unprotected Viterbi decoder, which comes from the calculation of PM_R during trace back. This does not affect the clock frequency. In the TMR implementation, no delay is introduced. Considering that the delay bottleneck of the receiver is usually the synchronization subsystem with much higher complexity, the one cycle extra delay in the decoder will not affect the throughput of the system.

B. Platform for SEUs Injection on Configuration Memory

To assess the effectiveness of the proposed scheme to protect against SEUs in the configuration memory, errors have been injected using an adapted version of the fault injection tool presented in [28] and implemented on a Zedboard. The overall experimental setup for injection on the unprotected and protected versions of the Viterbi decoder is shown in Fig. 7. The injection platform consists of two parts denoted by Programmable System (PS) and the Programmable Logic (PL). The PS consists of the ARM Cortex-A9 processor and dedicated fixed silicon controllers for different peripherals e.g., DDR memory controller, SD controller, UART controller etc. The DDR controller is responsible for storing the fault list (i.e., the list of configuration memory bits that needs to be evaluated). This fault list is generated during compile time from Xilinx generated design databases. This fault injection list is used by the injection algorithm for reliability evaluation of Viterbi decoders. The UART module in the PS is responsible for logging results to the PC. The PL part consists of two copies of the Viterbi Decoder i.e., Golden and Design Under Test (DUT). Moreover, a comparator and a synchronizer block are also present in the PL part. These modules are responsible for concurrent error detection in the Golden and DUT Viterbi copies and communicating the results to the PS part. It should be noted that the outputs of the Viterbi decoders are directly compared in hardware and only the mismatches are communicated to the ARM processor. Furthermore, the Viterbi decoder receives inputs from the ARM processor in the PS part. Therefore, the test patterns application is totally software-controlled. Similarly, the errors are counted in software running on the ARM processor based on the events generated by the synchronizer module in PL. The synchronizer module is also responsible for controlling the clock to DUT and Golden copies of Viterbi Decoder. Another important module

TABLE VIII
NUMBER OF ESSENTIAL BITS IN CONFIGURATION MEMORY

Design	HIVD	SIVD
Unprotected	380251	753918
DWC	843882	1646932
TMR	1236620	2385718

TABLE IX
FAILURE RATES OF THE PROTECTED VITERBI DECODER WITH HARD INPUT

BER	0	6.67%	10%	20%
Unprotected	4.7%	12.6%	13.9%	32%
DWC	0.18%	0.28%	0.30%	0.31%
TMR	0.14%	0.17%	0.18%	0.24%

TABLE X
FAILURE RATES OF THE PROTECTED VITERBI DECODER WITH SOFT INPUT

SNR	20 dB	10 dB	0 dB
Unprotected	7.4%	7.6%	11.6%
DWC	0.20%	0.21%	0.28%
TMR	0.07%	0.08%	0.11%

housed by the PL part is the Internal Configuration Access Port (ICAP) module, which allows the ARM processor to access the configuration memory related to the DUT decoder and modify it in run-time for error injection or removal. The modules in the PL region are connected to the PS region through AXI buses.

The ARM processor runs the software that controls the fault injection process. The fault injection starts by freezing the clock to the Viterbi decoder in the PL part (through the synchronizer module). This is followed by reading back the configuration memory through the ICAP port. The address of the configuration memory bits for fault injection is extracted from the fault list stored in DDR memory. The readback values are corrupted by inserting a bit flip for SEU emulation and written back. This is followed by resuming the design's clock. The inputs are generated in the same way as that for unprotected decoder in Section III-B. The software would collect two types of failures. One is that the output is provided but wrong (*Wrong Output*), and the other is that there is no output because error is detected but the faulty branch cannot be identified (*No Output*). The ARM processor keeps count of the two types of failures and communicates them to the PC for logging.

C. Tolerance to SEUs on Configuration Memory

The numbers of essential bits for configuration memories of HIVD and SIVD with DWC or TMR protection are reported in Table VIII, and the failure rate obtained with the fault injections are summarized in Table IX for HIVD and in Table X for SIVD, respectively. The values for the unprotected decoder are also provided as reference in these tables for comparison. Tables XI and XII show the fault types of DWC and TMR protected HIVD and SIVD, respectively.

From Tables IX and X, we can have the following observations: 1) the failure rate of the protected (DWC or TMR) is decreased by about two orders of magnitude relative to that

TABLE XI
FAILURE TYPES OF THE PROTECTED VITERBI DECODER WITH HARD INPUT
(*Wrong Output / No Output*)

BER	6.67%	10%	20%
DWC	2301/87	2477/89	2568/87
TMR	1534/584	1541/638	1605/1398

TABLE XII
FAILURE TYPES OF THE PROTECTED VITERBI DECODER WITH SOFT INPUT
(*Wrong Output / No Output*)

SNR	20 dB	10 dB	0 dB
DWC	3233/51	3339/53	4557/56
TMR	1391/376	1385/427	1579/1114

of unprotected decoders; 2) DWC provides similar SEU tolerance capability to TMR if the saving of resources is taken into consideration; 3) The SEU tolerance capability of DWC and TMR degrade a little for higher BER or lower SNR. Observations 1) and 2) show the effectiveness of the proposed technique. The reason that DWC provides slightly higher failure rate than TMR can be explained by the fact that its fault detection and correction module is more complex than that in the TMR scheme.

The reason for the third observation could be found in the results in Tables XI and XII. From the two tables we can see that the increment of failures for the DWC scheme is mainly from *Wrong Output* faults, which can be explained as follows. In the DWC structure, the PM for state 0 of the two decoders would be used to identify the faulty branch if one of them outputs wrong results. However, for high BER or low SNR that exceeds the error correction capability of the decoder, the path may not terminate at state 0 as expected, so the rule that the PM for the right path should be larger may not hold any more and the path selected (corresponding to the outputs) based on this rule may be wrong. Instead for the TMR scheme, the failure increment for higher BER or lower SNR mainly comes from *No Output* faults. Further analysis reveals that over 85% of *No Output* faults belong to the case that all three outputs are different and one of them is correct, which means a common configuration bit shared by two decoders is upset and the effect of the SEU is different in the two outputs. Such bits may be related to common control signals of different modules or registers in the same slice but used in two decoders. The logic or register may be related to the PM calculation for a state. The number of such shared control bits should be very small and fixed, so the effect of such fault to the final PM is fixed. However, whether the affected PM would change the path selection and further the decoding result depends on the difference between the PM of correct path and that of competing path. For an input signal with higher quality, the difference is larger so the probability that the fault could cause a wrong path selection is lower. In contrast, the *No Output* faults in the DWC scheme are caused when the output enabling signal in the fault detection and correction module is not activated, so the number of related bits is small and almost unchanged with signal quality.

TABLE XIII
OVERHEAD COMPARISON WITH [25] (FOR DECODER WITH SOFT INPUT)

	Unprotected	SO+EDAC+ P-TMR [25]	DWC	DWC +SO
LUTs	4091	6669	8672	6812
Registers	2614	4550	5460	4378
BRAM	1	2	2	2

D. Tolerance to SEUs on User Memory

Although the DWC scheme is proposed to protect against SEUs on the configuration memory, it should be also effective to SEUs on user memory. The effectiveness is evaluated by SEU injection experiments based on the same platform for decoder with soft inputs. The only modification is to change the list of configuration memory bits to that of 9300 user memory bits, among which 5460 bits are used as registers and $64 \times 60 = 3840$ bits are used in the BlockRAM for storing of surviving path information. Each bit in the fault list is upset for every one of the 122 cycles, and 100 input sequences are fed to the decoder to test the effect of a SEU on the current bit at the current cycle, so totally 100×122 SEU injections are performed for each bit. This process is repeated for SNRs of 0 dB, 10 dB and 20 dB. The results show that no SEU would cause wrong decoding results to the DWC protected decoder. In fact, as reported in [26], over 95% of SEUs on user memory in the Viterbi decoder could be tolerated without any protection. Therefore, the DWC scheme just protects the remaining 5% user memory bits against SEUs.

E. Comparison With Related Works

Reference [25] is the most relevant work, in which SEUs on both the configuration memory and the user memory of a SRAM FPGA implemented Viterbi decoder are considered. The comparison with [25] is done in three aspects as follows:

1) *Key Ideas*: The basic approach of [25] is to apply EDAC to protect Block RAM, and structure optimization (SO) to decrease the complexity of the C&S logic and BM/PM calculation so that the fault probability for these two parts is decreased. Then, only the important module of EDAC logic and the control logic are protected by TMR (Partial TMR, P-TMR). In contrast, the proposed DWC scheme is designed based on the fault analysis of Viterbi decoder, and the decoder is protected as whole. The advantage of the latter is that the effectiveness could be guaranteed in theory, and no modification is made to the inner parts of the decoder, which means that scheme is easy to apply in practice.

2) *Resource Efficiency*: The resource consumption of unprotected and protected decoder with different schemes are listed in Table XIII. As reported in [25], the overhead of the SO+EDAC+P-TMR protected decoder is about 1.6~1.7 times of the unprotected version, which is smaller than that of DWC scheme. However, if the SO method is applied in our implementation of the decoder, the overhead of the two main modules (C&S logic and PM/ BM calculation) can be decreased by about 21% and 23%, respectively, which makes the overhead of the DWC+SO protected decoder very closed to that of the SO+ EDAC+P-TMR scheme.

3) *Reliability*: Assuming a SEU rate per bit of 1.266×10^{-6} upsets/day, the reliability of the SO+EDAC+P-TMR scheme in [25] is predicted by calculation, and concludes that the failure rate of the whole decoder is 0.1132 failures/day. Based on the experiment results in Section V-C and V-D, failures of DWC protected decoder can only come from SEUs on configuration memory. With the same assumption, the failure rate of the DWC protected Viterbi decoder for hard input and soft input can be approximately estimated using the number of essential bits in Table VIII as $843882 \times 1.266 \times 10^{-6} \times 0.3\% = 0.0032$ failures/day and $1646932 \times 1.266 \times 10^{-6} \times 0.3\% = 0.0063$ failures/day, respectively, which are about 1/35 and 1/18 of that of the scheme from [25]. Therefore, the proposed scheme achieves a much better reliability.

VI. CONCLUSIONS

This paper has studied the effects of soft errors on the configuration memory of a Viterbi decoder implemented on an SRAM-FPGA. To that end, errors have been injected in the configuration memory, and the results show that the decoder is able to correct most of the errors when the BER is low or the SNR is high, but this immunity degrades significantly when the input signal deteriorates. Based on this first result, a new technique has been proposed to protect the Viterbi decoder against errors in the configuration memory. The scheme is based on an enhanced Duplication with Comparison (DWC) on which some internal signals are used to detect the copy in error so that upon a difference on the outputs, the ones from the correct copy can be used. The technique has been implemented and evaluated. The results show that the technique provides an efficient protection with a resource usage that is significantly lower than that of TMR. Compared to the scheme in [25], the proposed technique achieves a much better reliability with a similar overhead.

Future work will consider the development of a Concurrent Error Detection (CED) scheme using the internal signals of the decoder for error detection. Then upon error detection, the decoder functionality can be recovered by reconfiguring the FPGA. This approach can lower the resource usage of the protection at the cost of leaving the decoder not operational until the FPGA is reconfigured and the decoder is restarted.

REFERENCES

- [1] A. J. Viterbi, "Error bounds for convolutional codes and an asymptotically optimum decoding algorithm," *IEEE Trans. Inf. Theory*, vol. IT-13, no. 2, pp. 260–269, Apr. 1967.
- [2] S. Lin and D. J. Costello, *Error Control Coding: Fundamentals and Applications*, 2nd ed. Upper Saddle River, NJ, USA: Prentice-Hall, 2004.
- [3] J. G. Proakis and M. Salehi, *Fundamentals of Communication Systems*. Upper Saddle River, NJ, USA: Prentice-Hall, 2005.
- [4] J. G. Proakis, *Digital Communication*, 4th ed. New York, NY, USA: McGraw-Hill, 2002.
- [5] H. L. Lou, "Implementing the Viterbi algorithm," *IEEE Signal Process. Mag.*, vol. 12, no. 5, pp. 42–52, Sep. 1995.
- [6] N. Kanekawa *et al.*, *Dependability in Electronic Systems: Mitigation of Hardware Failures, Soft Errors, and Electro-Magnetic Disturbances*. New York, NY, USA: Springer-Verlag, 2010.
- [7] J. M. Aghadavoodi and K. Jakobs, "Concept of on-board-processing satellites," in *Proc. 1st Int. Conf. Universal Pers. Commun.*, 2002, pp. 14.04/1–14.04/4.

- [8] J. Louis Ippolito Jr., *Satellite Communication Systems Engineering*. Hoboken, NJ, USA: Wiley, 2008.
- [9] K. Maine, C. Devieux, and P. Swan, "Overview of IRIDIUM satellite network," in *Proc. WESCON*, San Francisco, CA, USA, Nov. 1995, pp. 483–490.
- [10] D. A. Sunderland *et al.*, "Megagate ASICs for the Thuraya satellite digital signal processor," in *Proc. Int. Symp. Quality Electron. Des.*, San Jose, CA, USA, Mar. 2002, pp. 479–486.
- [11] F. L. Kastensmidt, L. Carro, and R. Reis, *Fault-Tolerance Techniques for SRAM-Based FPGAs*. New Haven, CT, USA: Springer, 2006.
- [12] F. H. Robert, K. Somervill, and J. Williams, "An architecture for reconfigurable computing in space," in *Proc. Mil. Aerosp. Appl. Programmable Devices Technol. Conf.*, Washington, DC, USA, 2005, pp. 1–6.
- [13] T. Kuwahara, "FPGA-based reconfigurable on-board computing systems for space applications," *Ph.D. dissertation*, Inst. Space Syst., Universitat Stuttgart, Stuttgart, Germany, 2009.
- [14] S. Lopez, T. Vladimirova, C. Gonzalez, J. Resano, D. Mozos, and A. Plaza, "The promise of reconfigurable computing for hyperspectral imaging onboard systems: A review and trends," *Proc. IEEE*, vol. 101, no. 3, pp. 698–722, Mar. 2013.
- [15] P. S. Ostler *et al.*, "SRAM FPGA reliability analysis for harsh radiation environments," *IEEE Trans. Nucl. Sci.*, vol. 56, no. 6, pp. 3519–3526, Dec. 2009.
- [16] M. Caffrey *et al.*, "Single-event upsets in SRAM FPGAs," in *Proc. Mil. Aerosp. Appl. Programmable Devices Technol. Conf.*, Washington, DC, USA, 2002, pp. 1–6.
- [17] C. Carmichael, "Triple module redundancy design techniques for Virtex FPGAs," Xilinx, Corporation, Tech. Rep. XAPP197 (v1.0.1), Jul. 6, 2006.
- [18] J. Jonathan and W. Michael, "Voter insertion algorithms for FPGA designs using triple modular redundancy," in *Proc. 18th Annu. ACM/SIGDA Int. Symp. Field Programmable Gate Arrays*, Monterey, CA, USA, Feb. 21–23, 2010, pp. 249–258.
- [19] A. Rizwan *et al.*, "Design-for-diversity for improved fault-tolerance of TMR systems on FPGAs," in *Proc. Int. Conf. Reconfigurable Comput. FPGAs*, Nov. 30–Dec. 2, 2011, pp. 99–104.
- [20] P. L. Murray and D. Vanburen, "Single event effect mitigation in reconfigurable computers for space applications," in *Proc. IEEE Aerosp. Conf.*, 2005, pp. 1–7.
- [21] A. Reddy and P. Banarjee, "Algorithm-based fault detection for signal processing applications," *IEEE Trans. Comput.*, vol. 39, no. 10, pp. 1304–1308, Oct. 1990.
- [22] J. Geldmacher and J. Gotze, "On fault tolerant decoding of turbo codes," in *Proc. 7th Int. Symp. Turbo Codes Iterative Inf. Process.*, 2012, pp. 245–249.
- [23] J. Andrade *et al.*, "On the performance of LDPC and turbo decoder architectures with unreliable memories," in *Proc. 48th Asilomar Conf. Signals, Syst., Comput.*, 2014, pp. 542–547.
- [24] A. M. A. Hussien *et al.*, "A combined channel and hardware noise resilient Viterbi decoder," in *Proc. 44th Asilomar Conf. Signals, Syst., Comput.*, Pacific Grove, CA, USA, 2010, pp. 395–399.
- [25] Y. Wang *et al.*, "An SEU-tolerant approach for space-borne Viterbi decoders," *Chin. J. Electron.*, vol. 23, no. 4, pp. 857–861, Oct. 2014.
- [26] Z. Gao, L. Yan, J. Zhu, R. Han, and P. Reviriego, "Analysis of the effects of single event upsets (SEUs) on user memory in FPGA implemented Viterbi decoders," in *Proc. 31st IEEE Int. Symp. Defect Fault Tolerance VLSI Nanotechnol. Syst.*, Chicago, IL, USA, Oct. 8–10, 2018, pp. 1–6.
- [27] "Technical specification group radio access network, E-UTRA, Multiplexing and channel coding (Release 11)," 3rd Gener. Partnership Project, Sophia Antipolis, France, TS 36.212 V11.4.0 (2013-12).
- [28] A. Ullah *et al.*, "An FPGA based dynamically reconfigurable platform for emulation of permanent faults in ASICs," *Microelectron. Rel.*, vol. 75, pp. 110–120, Aug. 2017.
- [29] R. Le, "Soft error mitigation using prioritized essential bits," Xilinx, Inc., San Jose, CA, USA, Appl. Note, FPGAsXAPP538 (v1.0), Apr. 4, 2012.
- [30] J. Hussein and G. Swift, "Mitigating single-event upsets," Xilinx, Inc., San Jose, CA, USA, White Paper WP395 (v1.1), May 19, 2015.
- [31] P. Reviriego, C. Bleakley, and J. A. Maestro, "Diverse double modular redundancy: A new direction for soft error detection and correction," *IEEE Des. Test Comput.*, vol. 30, no. 2, pp. 87–95, Mar./Apr. 2013.

Authors' biographies not available at the time of publication.